Project Title

Smart Parking Management System (SPMS)

A Java-Based Application for Parking Slot Reservation and Management

---

Team Members

Name	Registration Number
Nahid Gazi	2022331048
Priom Chakraborty	2022331054
Ridoy Baidya	2022331070
Md.Mahdiul Hasan	2022331076
MD Rahat Ahmed	2022331096

Project Description

The **Smart Parking Management System (SPMS)** is a Java-based software application designed to solve common parking problems in urban areas, universities, offices, and commercial buildings. In traditional parking systems, drivers often spend a significant amount of time searching for vacant parking slots, which leads to traffic congestion, fuel wastage, and frustration.

This system provides **real-time parking slot availability** and allows users to **reserve parking spaces in advance**. It automatically records vehicle entry and exit times and calculates parking fees based on the duration of parking. Administrators can manage parking slots, monitor parking activity, and generate basic usage reports.

The project aims to demonstrate the practical application of **software engineering principles**, **object-oriented programming**, and **database management concepts** using Java.

Objectives

- To provide real-time information about available parking slots
- To allow users to reserve parking spaces easily
- To automate parking fee calculation based on parking duration
- To enable administrators to manage parking operations efficiently
- To reduce traffic congestion and improve parking efficiency
- To apply software engineering concepts learned in the course

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

Smart Parking Management System

(SPMS)

Document Title:	SRS for Smart Parking Management System
Version:	1.0
Date:	April 2026
Status:	Final Draft
Prepared By:	Development Team
Course:	Software Engineering

Table of Contents

1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Definitions and Abbreviations 3
1.4 References 4
2. Overall Description 4
2.1 Product Perspective 4
2.2 Product Functions 4
2.3 User Classes and Characteristics 4
2.4 Operating Environment 5
2.5 Design Constraints and Assumptions 5
3. Stakeholder Analysis 5
4. Functional Requirements 6
4.1 User Authentication 6
4.2 Parking Slot Management 6
4.3 Fee Calculation and Billing 7
4.4 Administrative Functions 7
4.5 Reporting and Analytics 7
5. Non-Functional Requirements 8
6. System Features 8
7. External Interface Requirements 9
8. UML Diagrams 10
8.1 Use Case Diagram 10
8.2 Class Diagram 11
9. Database Design 12
10. Business Rules 13
Appendix A: Glossary 14
Appendix B: Revision History & Approval 15

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the Smart Parking Management System (SPMS). It is prepared for academic course purposes and serves as a reference for the development team, course supervisor, and evaluators. The document is structured in accordance with IEEE Std 830-1998 guidelines.

1.2 Scope

The SPMS is a Java-based desktop application designed for use in a university or institutional setting. It enables vehicle owners to search for and reserve parking slots, and allows administrators to manage the parking facility and generate reports. The system handles real-time slot availability, reservations, vehicle check-in/check-out, automated fee calculation, and basic analytics.

This version is limited to a desktop implementation. Web and mobile variants are out of scope.

1.3 Definitions and Abbreviations

Term / Acronym	Definition
SPMS	Smart Parking Management System
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
GUI	Graphical User Interface
JDBC	Java Database Connectivity — Java API for database access
MVC	Model-View-Controller — the software architectural pattern used
Admin	System Administrator — a privileged user who manages the system
Slot	A single, uniquely numbered parking space within a parking lot
PK / FK	Primary Key / Foreign Key — database identifier terms
RBAC	Role-Based Access Control — access determined by the user's role
JRE / JDK	Java Runtime Environment / Java Development Kit

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Java SE 11 Documentation — Oracle Corporation
- MySQL 8.0 Reference Manual — Oracle Corporation
- Pressman, R. S. — Software Engineering: A Practitioner's Approach, 8th Ed.

2. Overall Description

2.1 Product Perspective

The SPMS is a standalone desktop application that connects to a local relational database for persistent storage. It follows a three-tier architecture: (1) Presentation Layer — the Java GUI; (2) Business Logic Layer — reservation, fee, and validation logic; (3) Data Access Layer — JDBC-based database operations. The system operates independently without external web service dependencies.

2.2 Product Functions

The major functions provided by the SPMS are summarized below:

- User registration, login, and profile management
- Real-time visual display of parking slot availability (colour-coded grid)
- Advance slot reservation with date, time, and duration selection
- Vehicle check-in and check-out with automatic timestamp recording
- Automated fee calculation using a tiered pricing model
- Digital receipt generation and transaction history storage
- Administrator dashboard for lot/slot/user management
- Reporting module with utilization, revenue, and peak-hour analytics

2.3 User Classes and Characteristics

User Type	Description	Technical Level	Primary Actions
End User (Driver)	General public using the parking facility for their vehicle	Basic computer literacy	Register, search slots, reserve, check-in, check-out, view history
Administrator	Facility manager with elevated system privileges	Moderate technical knowledge	Manage lots/slots/users, configure pricing, generate reports

2.4 Operating Environment

- Operating System: Windows 10/11, macOS 10.14+, or Ubuntu 20.04+
- Java Runtime Environment (JRE): Version 11 or higher
- Database: MySQL 8.0+ or SQLite 3.0+
- Hardware: Intel Core i3+, 4 GB RAM minimum, 500 MB free disk space, 1024×768 display
- Network: Optional — required only for remote multi-user database access

2.5 Design Constraints and Assumptions

- The system must be implemented in Java with JDBC database connectivity (academic requirement)
- GUI must use JavaFX or Swing; MVC pattern is required
- All users are assumed to possess basic computer operation skills
- The parking facility is assumed to have uniquely numbered, pre-configured physical slots
- The host system clock is assumed accurate for timestamp and billing calculations

3. Stakeholder Analysis

Stakeholders are all individuals or groups with a direct or indirect interest in the SPMS. The table below identifies each stakeholder, their type, and their primary expectations from the system.

Stakeholder	Type	Key Interests and Responsibilities
Vehicle Owner (Driver)	Primary	Searches for available slots, makes reservations, checks in/out, views billing history. Needs a fast and easy-to-use interface.
Parking Administrator	Primary	Manages parking lots, slots, and user accounts. Monitors occupancy in real time. Generates and reviews operational reports. Requires full system control.
System Developer	Secondary	Designs, builds, and tests the system. Requires clear, unambiguous requirements and a maintainable modular architecture.
Institution / Organization	Secondary	The deploying body (e.g., university, office, mall). Expects accurate reporting, revenue tracking, system reliability, and future scalability.

Understanding these stakeholders ensures that both the functional and non-functional requirements are aligned with real-world needs. The primary stakeholders drive the core feature set; the secondary stakeholders influence quality attributes such as reliability, scalability, and maintainability.

4. Functional Requirements

4.1 User Authentication and Authorization

ID	Requirement Description	Priority
FR-1.1	The system shall allow new users to register by providing a unique username, password, email, phone number, vehicle type, and vehicle registration number.	High
FR-1.2	The system shall validate email format and phone number format during registration; invalid formats shall be rejected with an inline error message.	High
FR-1.3	All passwords shall be stored as hashed values (e.g., BCrypt). Plaintext passwords shall never be stored or logged.	High
FR-1.4	The system shall authenticate users with their username and password, then redirect them to their role-specific dashboard.	High
FR-1.5	After three consecutive failed login attempts, the account shall be locked for 15 minutes.	High
FR-1.6	An inactive session shall automatically time out after 30 minutes, returning the user to the login screen.	Medium
FR-1.7	The system shall enforce RBAC with two roles: End User and Administrator. Each role may only access its designated modules.	High

4.2 Parking Slot Management

ID	Requirement Description	Priority
FR-2.1	The system shall display a real-time visual grid of all parking slots. Available slots appear green, reserved slots yellow, occupied slots red, and out-of-service slots grey.	High
FR-2.2	Users shall be able to reserve an available slot by selecting a parking lot, date, start time, and duration (30-minute minimum increments).	High
FR-2.3	The system shall prevent double-booking: any reservation that overlaps an existing booking on the same slot shall be rejected.	High
FR-2.4	Users shall be able to cancel a reservation. Cancellations made more than 1 hour before the start time are free; cancellations within 1 hour incur a 50% fee.	High
FR-2.5	The system shall record the check-in timestamp when a user marks arrival, changing slot status from Reserved to Occupied.	High
FR-2.6	On check-out, the system shall record the exit timestamp, calculate total duration and fee, generate a digital receipt, and change slot status to Available.	High
FR-2.7	Users shall be able to view all upcoming reservations and their complete booking history.	Medium

4.3 Fee Calculation and Billing

ID	Requirement Description	Priority
FR-3.1	The system shall calculate fees based on a tiered rate: $2.00/hour for the first 3 hours, $1.50/hour thereafter, with a daily cap of $15.00.	High
FR-3.2	Vehicle-type multipliers shall apply: Standard = 100%, Motorcycle = 50%, Large vehicle (SUV/truck) = 150% of the applicable rate.	High
FR-3.3	Partial hours shall be rounded up to the nearest full hour for billing.	Medium
FR-3.4	A digital receipt shall be generated on check-out, including: user name, vehicle details, slot number, entry/exit times, duration, and total fee.	High

4.4 Administrative Functions

ID	Requirement Description	Priority
FR-4.1	Administrators shall be able to add, edit, and deactivate parking lots (name, location, total capacity).	High
FR-4.2	Administrators shall be able to add parking slots to a lot, specifying slot number, type (Standard/Motorcycle/Large), and initial status.	High
FR-4.3	Administrators shall be able to mark individual slots as Out-of-Service, removing them from the reservation pool.	Medium
FR-4.4	Administrators shall be able to view, activate, deactivate, and reset passwords for all user accounts.	High
FR-4.5	Administrators shall be able to configure system pricing parameters (rates, multipliers, daily cap).	Medium

4.5 Reporting and Analytics

ID	Requirement Description	Priority
FR-5.1	The system shall generate parking utilization reports showing total reservations, occupancy rate, and average duration, filterable by date range and parking lot.	High
FR-5.2	The system shall generate revenue reports with daily, weekly, and monthly income breakdowns by parking lot.	High
FR-5.3	The system shall identify and display peak usage hours and occupancy patterns by day of week.	Medium
FR-5.4	All reports shall be exportable to PDF format.	Medium

5. Non-Functional Requirements

ID	Category	Requirement	Metric / Target
NFR-1.1	Performance	The system shall respond to any user action within 2 seconds under normal load.	Response time < 2 sec for 100 concurrent users
NFR-2.1	Security	Passwords shall be stored exclusively as cryptographic hashes; SQL injection shall be prevented through parameterized queries.	Zero plaintext passwords in DB; no raw SQL concatenation
NFR-2.2	Security	RBAC shall prevent any unauthorized access to admin-only modules.	Penetration test confirms no privilege escalation
NFR-3.1	Usability	All primary user tasks shall be completable by a first-time user within 5 minutes without training.	Usability test — avg task time ≤ 5 min for 5 novice users
NFR-4.1	Reliability	The system shall maintain minimum 99% uptime during scheduled operating hours.	Monitoring logs over 30 days confirm ≥ 99% availability
NFR-4.2	Reliability	The system shall support database backup and point-in-time recovery.	Recovery test completes successfully within 30 minutes
NFR-5.1	Maintainability	All public classes and methods shall have Javadoc documentation. Min. 70% unit test coverage required.	Javadoc builds without errors; JaCoCo ≥ 70% line coverage
NFR-6.1	Portability	The system shall run without modification on Windows, macOS, and Linux with JRE 11+.	Verified on all three target OS environments
NFR-7.1	Scalability	The business logic layer shall be sufficiently decoupled to support future web or mobile frontend integration.	Architecture review confirms interface-layer isolation

6. System Features

6.1 User Authentication Module

Handles registration, secure login, session management, account lockout, and role-based access control. Passwords are hashed using BCrypt and sessions time out after 30 minutes of inactivity.

6.2 Parking Slot Management Module

Manages the full lifecycle of parking slots — from initial configuration to real-time availability tracking. A colour-coded visual grid is the primary interface for both users and administrators. Status changes are triggered automatically by reservations, check-ins, and check-outs.

6.3 Reservation System

Allows users to book slots up to 30 days in advance using a calendar-based form with 30-minute granularity. Conflict detection prevents double-booking. No-show reservations are auto-cancelled after a 30-minute grace period.

6.4 Automated Billing Module

Computes fees on check-out using tiered rates and vehicle-type multipliers, enforces the daily cap, and generates a detailed digital receipt stored in the user's transaction history.

6.5 Administrative Dashboard

A tabbed control panel giving administrators full management of lots, slots, users, and pricing. All actions are logged for audit purposes.

6.6 Reporting and Analytics Module

Aggregates transactional data to produce utilization and revenue reports. Supports custom date ranges and lot-level filtering. Reports are exportable to PDF.

7. External Interface Requirements

7.1 User Interface

The SPMS interface shall be clean and consistent, using a navy-blue and white colour scheme with status-based colour coding for slot indicators. All screens shall provide clear error feedback and conform to standard desktop UI conventions. Key screens include:

- Login Screen: Username/password fields, 'Remember Me' option, 'Forgot Password' link
- Registration Screen: All required fields with inline validation, vehicle type selector
- User Dashboard: Active reservation summary, quick-action buttons, recent activity feed
- Slot View: Full grid of all slots with live colour coding and filtering by lot/type/status
- Reservation Form: Date picker, time selector, duration input, conflict checking on submission
- Check-In/Check-Out Screen: Active reservation details, confirm buttons, receipt display on checkout
- Admin Panel: Tabbed interface — Lots | Slots | Users | Pricing | Reports
- Reports Screen: Date range pickers, lot selector, summary statistics, PDF export button

7.2 Hardware Interfaces

The SPMS is a pure software solution. All input is via standard keyboard and mouse. Output is rendered on a standard display (1024×768 minimum). There is no integration with physical hardware such as parking barriers, sensors, or payment terminals in this version.

7.3 Software Interfaces

Component	Details
Database System	MySQL 8.0+ (multi-user deployment) or SQLite 3.0+ (single-user / development)
Database Connectivity	JDBC API — MySQL Connector/J or SQLite JDBC driver
GUI Framework	JavaFX 11+ or Swing (included in JDK 11)
Runtime Environment	JRE 11 or higher; JDK 11+ required for development
File System	Read/write access for configuration files, logs, and PDF report output directory

7.4 Communication Interfaces

- MySQL database communication over TCP/IP on port 3306; SSL/TLS encryption recommended
- SQLite uses local file access — no network communication required
- All queries transmitted as parameterized SQL via JDBC; result sets returned as Java ResultSet objects

8. UML Diagrams

This section presents the key UML diagrams that describe the structural and behavioural design of the SPMS. These diagrams serve as the primary implementation blueprint for the development team.

8.1 Use Case Diagram

The Use Case Diagram identifies the system's two actors — the End User (Driver) and the Administrator — and maps them to the system's functional capabilities. The diagram shows the system boundary, actor-to-use-case associations, and key include/extend relationships.

Actors:

- User (Vehicle Owner / Driver) — external actor who consumes parking services
- Administrator — privileged actor who manages and monitors the system

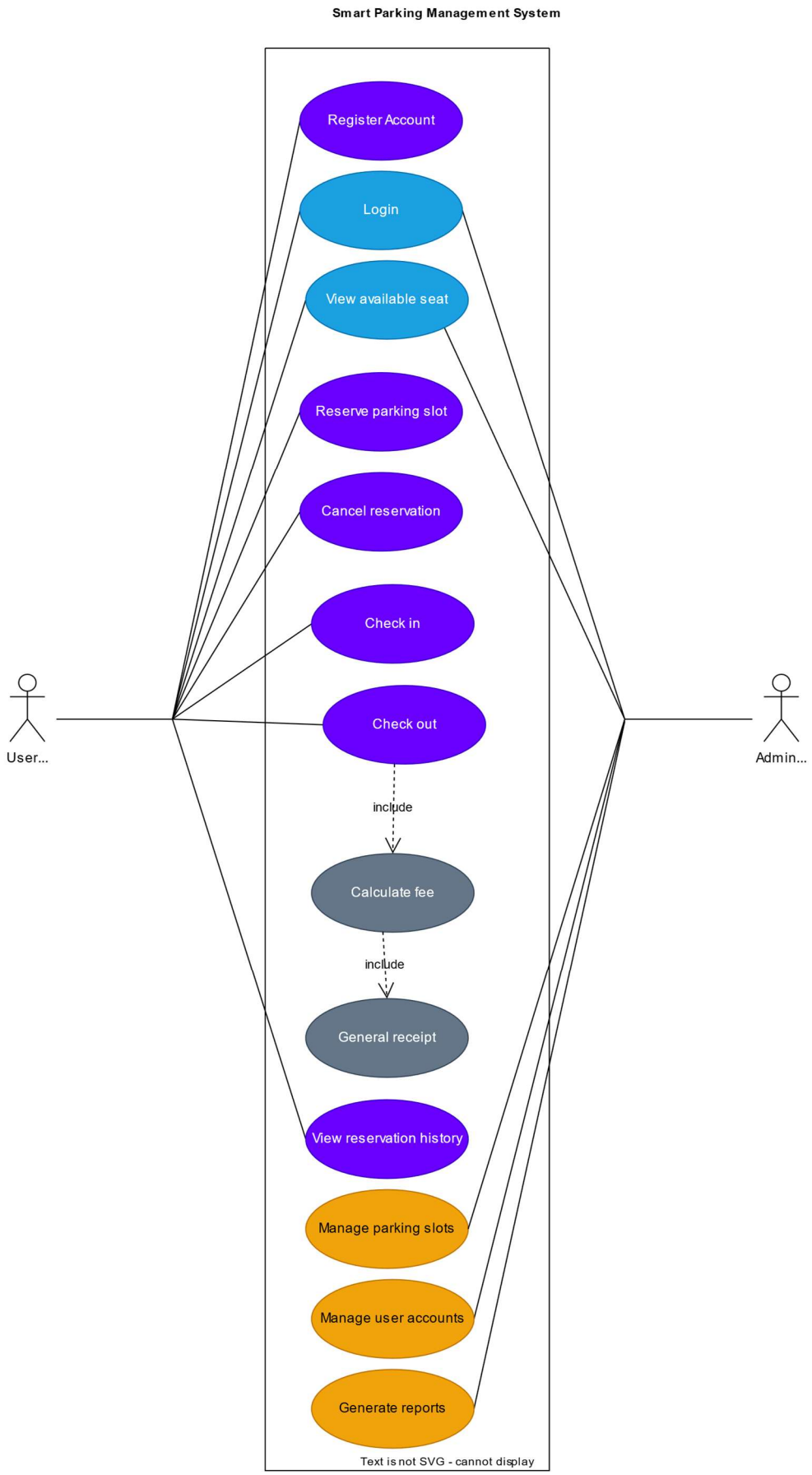
User Use Cases:

- Register Account
- Login
- View Available Parking Slots
- Reserve Parking Slot <<includes>> Check Slot Availability, Detect Conflicts
- Cancel Reservation <<extends>> Apply Cancellation Fee
- Check In
- Check Out <<includes>> Calculate Fee, Generate Receipt
- View Reservation History
- Update Profile / Change Password

Administrator Use Cases:

- Login
- Manage Parking Lots (Add / Edit / Deactivate)
- Manage Parking Slots (Add / Edit / Mark Out-of-Service)
- Manage User Accounts (View / Activate / Deactivate / Reset Password)
- Configure Pricing Rules
- Monitor Real-time Occupancy Dashboard
- Generate Utilization and Revenue Reports
- Export Reports to PDF

USE CASE DIAGRAM :


Smart Parking Management System
Register Account
Login
View available seat
Reserve parking slot
Cancel reservation
Check in
Check out
include
Calculate fee
include
General receipt
View reservation history
Manage parking slots
Manage user accounts
Generate reports
User...
Admin...
Text is not SVG - cannot display

8.2 Class Diagram

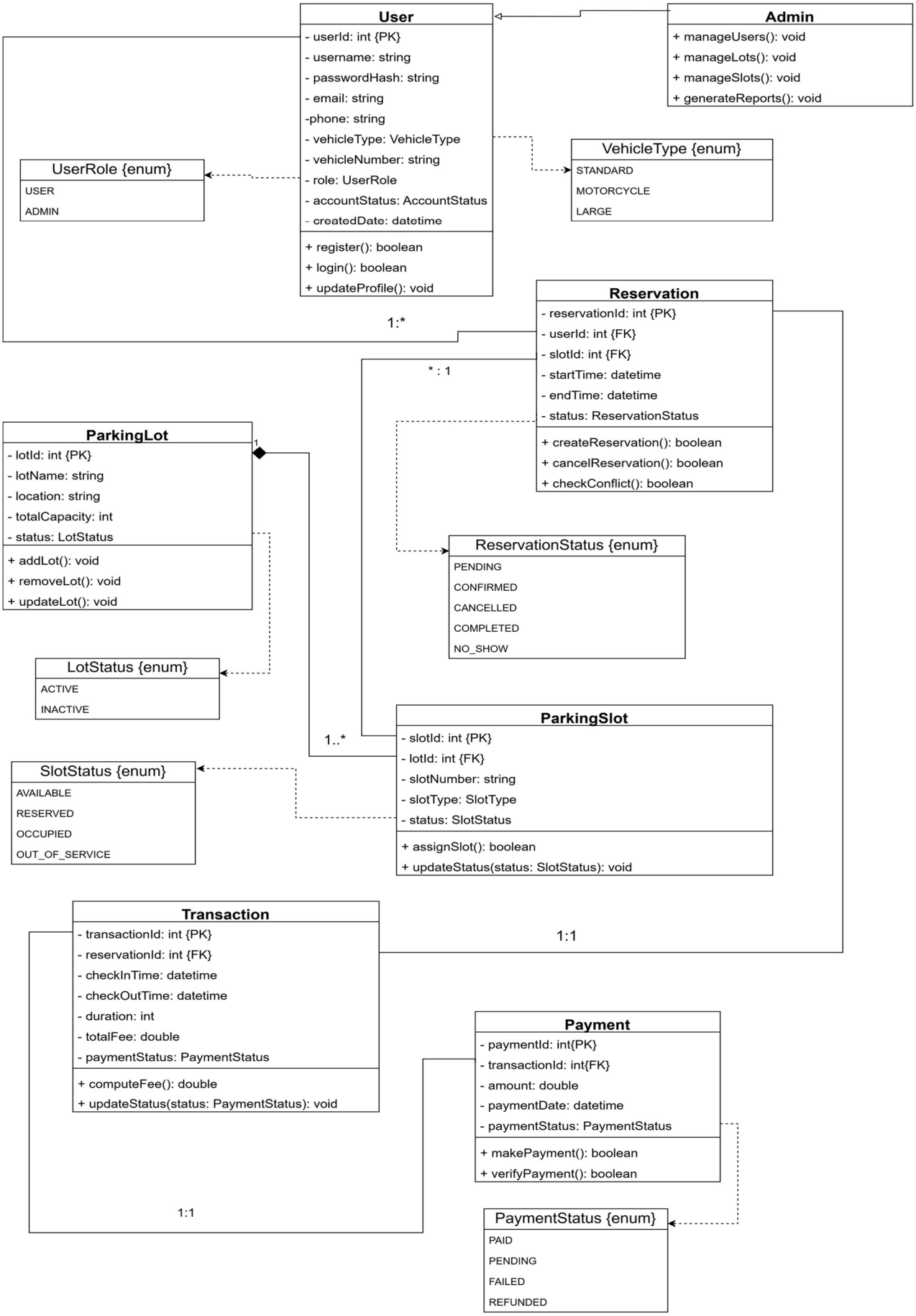
The Class Diagram represents the static object-oriented structure of the SPMS, showing all domain classes, their attributes, methods, and inter-class relationships. This diagram directly guides the implementation of the data model and business logic layers.

Class	Key Attributes	Key Methods
User	userId: int (PK) username, password (hashed) email, phone vehicleType, vehicleNumber role: {USER, ADMIN} accountStatus, createdDate	register(), login(), logout() updateProfile() changePassword() getReservations(): List
Admin	adminId: int (PK) username, password email	login() manageSlots(), manageLots() manageUsers() generateReport()
ParkingLot	lotId: int (PK) lotName, location totalCapacity: int status: {ACTIVE, INACTIVE}	addSlot(), removeSlot() getAvailableSlots(): List getOccupancyRate(): double
ParkingSlot	slotId: int (PK) lotId: int (FK) slotNumber: String slotType: {STANDARD, MOTORCYCLE, LARGE} status: {AVAILABLE, RESERVED, OCCUPIED, OUT_OF_SERVICE}	reserve(), checkIn(), checkOut() setOutOfService() isAvailable(): boolean
Reservation	reservationId: int (PK) userId: int (FK) slotId: int (FK) startTime, endTime: DateTime status: {PENDING, CONFIRMED, CANCELLED, COMPLETED, NO_SHOW}	confirm(), cancel() getDuration(): int checkForConflicts(): boolean
Transaction	transactionId: int (PK) reservationId: int (FK) checkInTime, checkOutTime: DateTime duration: int (minutes) totalFee: double paymentStatus	calculateFee(): double generateReceipt(): Receipt
Payment	paymentId: int (PK) transactionId: int (FK) amount: double paymentDate: DateTime paymentStatus	process(), refund() getReceipt(): String

Class Relationships:

- User (1) —— (0..*) Reservation: One user may have zero or many reservations
- ParkingSlot (1) —— (0..*) Reservation: One slot may appear in many sequential reservations
- ParkingLot (1) —— (1..*) ParkingSlot: A lot contains one or more slots
- Reservation (1) —— (0..1) Transaction: A completed reservation has at most one transaction
- Transaction (1) —— (0..1) Payment: Each transaction produces one payment record

CLASS DIAGRAM :


User
- userId: int {PK}
- username: string
- passwordHash: string
- email: string
-phone: string
- vehicleType: VehicleType
- vehicleNumber: string
- role: UserRole
- accountStatus: AccountStatus
- createdDate: datetime
+ register(): boolean
+ login(): boolean
+ updateProfile(): void
Admin
+ manageUsers(): void
+ manageLots(): void
+ manageSlots(): void
+ generateReports(): void
UserRole {enum}
USER
ADMIN
VehicleType {enum}
STANDARD
MOTORCYCLE
LARGE
1:*
Reservation
- reservationId: int {PK}
- userId: int {FK}
- slotId: int {FK}
- startTime: datetime
- endTime: datetime
- status: ReservationStatus
+ createReservation(): boolean
+ cancelReservation(): boolean
+ checkConflict(): boolean
* : 1
ParkingLot
- lotId: int {PK}
- lotName: string
- location: string
- totalCapacity: int
- status: LotStatus
+ addLot(): void
+ removeLot(): void
+ updateLot(): void
1
ReservationStatus {enum}
PENDING
CONFIRMED
CANCELLED
COMPLETED
NO_SHOW
LotStatus {enum}
ACTIVE
INACTIVE
1..*
ParkingSlot
- slotId: int {PK}
- lotId: int {FK}
- slotNumber: string
- slotType: SlotType
- status: SlotStatus
+ assignSlot(): boolean
+ updateStatus(status: SlotStatus): void
SlotStatus {enum}
AVAILABLE
RESERVED
OCCUPIED
OUT_OF_SERVICE
Transaction
- transactionId: int {PK}
- reservationId: int {FK}
- checkInTime: datetime
- checkOutTime: datetime
- duration: int
- totalFee: double
- paymentStatus: PaymentStatus
+ computeFee(): double
+ updateStatus(status: PaymentStatus): void
1:1
Payment
- paymentId: int{PK}
- transactionId: int{FK}
- amount: double
- paymentDate: datetime
- paymentStatus: PaymentStatus
+ makePayment(): boolean
+ verifyPayment(): boolean
1:1
PaymentStatus {enum}
PAID
PENDING
FAILED
REFUNDED

9. Database Design

9.1 Entity-Relationship Overview

The SPMS database is organized around six core entities. The table below defines each entity with its attributes and key constraints.

Entity / Table	Attributes	Key Constraints
Users	UserID (PK), Username, Password, Email, Phone, VehicleType, VehicleNumber, Role, AccountStatus, CreatedDate	PK on UserID; UNIQUE on Username, Email; NOT NULL on all fields
ParkingLots	LotID (PK), LotName, Location, TotalCapacity, Status, CreatedDate	PK on LotID; NOT NULL on LotName, Capacity
ParkingSlots	SlotID (PK), LotID (FK), SlotNumber, SlotType, SlotStatus, CreatedDate	PK on SlotID; FK to ParkingLots; UNIQUE on (LotID, SlotNumber)
Reservations	ReservationID (PK), UserID (FK), SlotID (FK), StartTime, EndTime, Status, CreatedDate	PK on ReservationID; FK to Users and ParkingSlots
Transactions	TransactionID (PK), ReservationID (FK), CheckInTime, CheckOutTime, Duration, TotalFee, PaymentStatus	PK on TransactionID; FK to Reservations
Payments	PaymentID (PK), TransactionID (FK), Amount, PaymentDate, PaymentStatus	PK on PaymentID; FK to Transactions

9.2 Entity Relationships

Relationship	Cardinality	Description
Users → Reservations	1 : Many	One user may create many reservations over time
ParkingSlots → Reservations	1 : Many	One slot may appear in many sequential (non-overlapping) reservations
ParkingLots → ParkingSlots	1 : Many	One parking lot contains one or more individual slots
Reservations → Transactions	1 : 1	Each completed reservation generates exactly one transaction record
Transactions → Payments	1 : 1	Each transaction is associated with exactly one payment entry

9.3 Data Integrity Constraints

- Primary keys defined on all tables to uniquely identify each record
- Foreign key constraints with referential integrity enforced on all inter-table relationships
- NOT NULL constraints applied to all critical fields (names, timestamps, status values)
- UNIQUE constraints prevent duplicate usernames, email addresses, and slot numbers within a lot
- CHECK constraints enforce valid enumerated values for all status and type fields
- On user account deletion, reservation and transaction history is archived, not permanently removed

10. Business Rules

10.1 Reservation Rules

- A user may hold at most one active reservation per parking lot at any given time
- Reservations may be created up to 30 calendar days in advance
- The minimum reservation duration is 30 minutes
- No-show reservations are automatically cancelled if the user does not check in within 30 minutes of the scheduled start time
- Cancellations made more than 1 hour before start time: no charge
- Cancellations made within 1 hour of start time: 50% cancellation fee applies

10.2 Pricing Rules

Rule	Details
Base hourly rate	$2.00 per hour for the first 3 hours of parking
Extended rate	$1.50 per hour for any duration beyond 3 hours
Daily maximum cap	$15.00 for any continuous 24-hour parking period
Partial hour rounding	Any partial hour is rounded up to the next full hour for billing
Motorcycle multiplier	50% of the applicable standard rate
Large vehicle multiplier	150% of the applicable standard rate (SUVs, trucks, vans)
Standard vehicle	100% of the applicable rate — no multiplier applied

10.3 Access Control Rules

- No unauthenticated user may access any system function beyond login and registration
- Role assignment is determined at account creation; users cannot self-escalate privileges
- End Users may only view and interact with their own data
- Administrators have unrestricted access to all modules and all user data

10.4 Data Retention and Audit Rules

- All financial transactions must be retained permanently and cannot be deleted from the system
- All administrative actions shall be logged with timestamps and the acting administrator's ID
- Deactivating a user account does not delete their reservation or transaction history

Appendix A: Glossary

Term / Acronym	Full Form / Definition
SPMS	Smart Parking Management System — the software system described in this SRS
SRS	Software Requirements Specification — this document
FR / NFR	Functional Requirement / Non-Functional Requirement
GUI	Graphical User Interface — the visual layer through which users interact with the software
JDBC	Java Database Connectivity — the standard Java API for connecting to relational databases
MVC	Model-View-Controller — the architectural pattern used to separate concerns in the application
JRE / JDK	Java Runtime Environment / Java Development Kit — required execution and build environments
PK / FK	Primary Key / Foreign Key — standard relational database identifier terms
RBAC	Role-Based Access Control — a security model where access is determined by the user's assigned role
BCrypt	A password hashing algorithm commonly used for secure credential storage in Java applications
Check-In	The act of recording a vehicle's arrival at the facility and activating the parking session
Check-Out	The act of recording departure, computing session duration, and generating the billing receipt
Occupancy Rate	The percentage of total parking slots that are currently in an occupied or reserved state
Peak Hours	Time intervals with historically the highest concentration of parking activity
No-Show	A reservation for which the user did not check in within the permitted grace period
Tiered Pricing	A fee structure where the per-hour rate changes based on the total duration of parking
Session	An active, authenticated user login period — subject to automatic 30-minute inactivity timeout
Out-of-Service Slot	A parking slot temporarily unavailable for reservation due to maintenance or other reasons

Appendix B: Revision History and Document Approval

B.1 Revision History

Version	Date	Author	Description of Changes
1.0	April 2026	Development Team	Initial SRS document created for course submission

B.2 Document Approval

This Software Requirements Specification has been reviewed and is approved by the following parties:

Name	Role	Signature / Date
Student Name	Author / Developer	
Student Name	Team Member	
Course Supervisor	Academic Evaluator	
Department Head	Final Approver	

— End of Document —